LOGIC MATRIX

contact@logicmatrix.tech
+1 703 8136414
www.logicmatrix.tech
Manassas, VA, 20112

DevOps Engineer Assessment

Build a Production-Ready Kubernetes Platform on Azure or AWS

Objective

Build a simple production-style platform using Docker, CI/CD, Kubernetes, Terraform, and either Azure AKS or AWS EKS.

The goal is to evaluate practical DevOps, cloud, Kubernetes, Terraform, CI/CD, troubleshooting, and security knowledge.

Submission Requirement

Submit all code, configuration, and documentation in a GitHub repository.

Repository can be public. If private, access must be provided for review.

Do not commit secrets, passwords, tokens, private keys, webhook URLs, cloud keys, or Terraform state files.

Task 1: Frontend, Backend, Docker, and Docker Compose

Create two separate applications:

- Frontend application
- Backend API application

Backend must include:

- `/` endpoint returning `Application is running`

- `/health` endpoint returning `{ "status": "ok" }`
- Must run on port `8080`

Frontend must:

- Run as a separate container
- Call the backend API
- Show a simple web page

Add:

frontend/
backend/
docker-compose.yml
.dockerignore

Both apps should run locally using:

docker compose up -d

Backend should be testable using:

curl http://localhost:8080
curl http://localhost:8080/health

Task 2: CI/CD Pipeline

Create one pipeline using one of the following:

- GitHub Actions
- Jenkinsfile
- Azure DevOps Pipeline

Pipeline must include:

- Checkout code
- Install dependencies
- Run tests
- Build frontend Docker image
- Build backend Docker image

- Tag both images
- Push images to ACR or ECR, or mock the push
- Create a GitHub release or release tag
- Deploy both apps to Kubernetes, or mock deploy

Explain how secrets should be stored safely using GitHub Secrets, Jenkins credentials, Azure DevOps variable groups, Azure Key Vault, or AWS Secrets Manager.

Task 3: Kubernetes Manifests

Create Kubernetes files under a `k8s/` folder.

Required files:

```
k8s/
  frontend-deployment.yaml
  frontend-service.yaml
  backend-deployment.yaml
  backend-service.yaml
  ingress.yaml or gateway.yaml
  backend-configmap.yaml
  backend-secret-example.yaml
```

Kubernetes requirements:

- Frontend and backend must be separate deployments
- Backend must run on port `8080`
- Minimum 2 replicas
- Readiness probe
- Liveness probe
- Resource requests and limits
- ConfigMap usage
- Secret example for database credentials
- Image tag must not be hardcoded as `latest`
- Ingress or Gateway must expose frontend externally
- Backend should be internal only

Task 4: Private Database Connectivity

Backend must connect to a database privately.

The database must not be publicly exposed.

Candidate may use:

- Azure SQL, Azure PostgreSQL, Azure MySQL, or VM-based database
- AWS RDS, PostgreSQL, MySQL, or EC2-based database

Candidate must explain:

- How AKS/EKS connects privately to the database
- Private subnet or private endpoint design
- Private DNS requirement
- NSG, firewall, or security group rules
- How only backend can access the database
- How database credentials are stored securely
- How to confirm database is not publicly accessible

Task 5: Terraform Cluster Provisioning

Create a `terraform/` folder showing how to provision and maintain AKS or EKS.

Required files:

```
terraform/
  provider.tf
  main.tf
  variables.tf
  outputs.tf
  README.md
  modules/
```

Terraform must be module-based.

Candidate can create custom modules only. Third-party or ready-made Terraform modules are not allowed.

Terraform should include or explain:

- Resource group or VPC
- Network and subnet design
- AKS or EKS cluster
- Node pool or node group
- ACR or ECR
- Monitoring: Log Analytics or CloudWatch
- Private database connectivity
- Remote backend and state locking
- Variables for environment, region, cluster name, node size, node count, and Kubernetes version
- Outputs for cluster name, endpoint, registry name, and network ID

Candidate must also explain:

- How to safely upgrade AKS/EKS
- How to add or resize node pools
- How to maintain Terraform state
- How to avoid downtime during cluster changes
- How to separate dev, staging, and production
- How to handle secrets outside Terraform code
- What to check if Terraform wants to recreate the cluster

Task 6: Troubleshooting Questions

Create:

docs/troubleshooting.md

Answer briefly:

1. Pod is in `CrashLoopBackOff`. What do you check?
2. Deployment is successful, but app is not reachable. What do you check?
3. Difference between readiness and liveness probe?
4. Docker build works locally but fails in pipeline. Why?
5. Pipeline fails during Docker build. What do you check?
6. Certificate renewal failed. What do you check?
7. Ingress returns `502` or `504`. What do you check?
8. Vendor SFTP connection to port `22` times out. What do you check?
9. Terraform plan wants to recreate the cluster. What do you check?

10. How would you upgrade AKS/EKS safely?
11. Frontend loads, but backend API calls fail. What do you check?
12. Backend pod is running, but database connection times out. What do you check?
13. Private DNS is not resolving database hostname. What do you check?
14. How would you rotate database credentials safely?
15. Secrets were accidentally committed to GitHub. What do you do?

Task 7: Future Improvement Proposal

Create:

docs/future-improvements.md

For each improvement, explain:

- What improvement is recommended
- Why it is needed
- How it helps the team or business
- How it would be implemented
- What risk it reduces

Example areas:

- Secret management
- Image vulnerability scanning
- Monitoring and alerting
- Rollback strategy
- Helm chart
- Terraform remote backend
- Kubernetes autoscaling
- Cluster upgrade strategy
- Production approval gates
- Private cluster
- WAF
- GitOps with Argo CD
- Blue/green or canary deployment
- Backup and disaster recovery
- Network policies
- Cost optimization

Expected GitHub Structure

```
devops-assessment/
  frontend/
    Dockerfile
    source-code-files

  backend/
    Dockerfile
    source-code-files

  docker-compose.yml
  .dockerignore

  .github/
    workflows/
      deploy.yml

  k8s/
    frontend-deployment.yaml
    frontend-service.yaml
    backend-deployment.yaml
    backend-service.yaml
    ingress.yaml or gateway.yaml
    backend-configmap.yaml
    backend-secret-example.yaml

  terraform/
    provider.tf
    main.tf
    variables.tf
    outputs.tf
    README.md
    modules/

  docs/
    troubleshooting.md
    future-improvements.md

  README.md
```

Evaluation Criteria

Submission will be reviewed based on:

- GitHub structure
- Frontend and backend separation
- Docker and Docker Compose quality
- CI/CD pipeline understanding
- Image tagging and ACR/ECR push
- Kubernetes manifest quality
- Private database connectivity design
- Terraform module-based structure
- AKS/EKS provisioning knowledge
- Terraform maintenance understanding
- Security and secret management
- Troubleshooting approach
- Documentation quality
- Future improvement explanation
- Production-readiness mindset

Note for Candidate

This assessment is designed to check the real working knowledge of DevOps/Cloud Engineer.

The main focus is not only whether the application runs, but how well you design, automate, secure, troubleshoot, and explain a production-style cloud platform.